

SOURCE LOG: src\config_forge\make_config.py

```
# -*- coding: utf-8 -*-
import os # Инструмент операционной системы
import sys # Управление потоками ввода-вывода APX
from pathlib import Path # Высокоуровневая навигация по дискам

def log(msg: str):
    """Строгий UTF-8 вывод в терминал космолуны"""
    sys.stdout.buffer.write(f"[FORGE_CORE] {msg}\n".encode('utf-8'))

def init_core_structure() -> tuple[Path, Path, Path]:
    """Разметка физических секторов и проверка бункера config_models"""
    root = Path(r"Z:\flowch")
    data = root / "dataset" / "mng oks bl"
    run_dir = root / "run"

    # Замечание №1: Изолированный бункер для хранения полетных листов
    config_dir = root / "src" / "config_models"
    config_dir.mkdir(parents=True, exist_ok=True)

    log(f"Бункер для YAML-конфигов развернут: {config_dir}")
    return root, data, config_dir

def build_config_text(run_dir: Path, data_dir: Path) -> str:
    """Сборка текстового массива YAML-конфига"""
    log("Начало генерации структуры полетного листа...")

    # Эшелон метаданных задания и параметров матрицы
    yaml_lines = [
        "job: extension",
        "config:",
        "  name: \"chroma_mng oks v1\" # Имя папки сохранения",
        "  process:",
        "    - type: 'sd_trainer' # Движок плавки",
        f"      training_folder: \"{run_dir}\" # Куда лить веса",
        "      device: cuda:0 # Топливо — первая RTX 3090",
        "      network:",
        "        type: \"lora\" # Архитектура адаптера",
        "        linear: 16 # Ранг матрицы (Rank)",
        "        linear_alpha: 16 # Альфа-множитель для баланса",
    ]
    return "\n".join(yaml_lines) + "\n"

def append_dataset_and_train_params(current_yaml: str, data_dir: str) -> str:
    """Интеграция старого датасета и лимитов для RTX 3090"""
    # Дописываем сектора датасета и параметров оптимизации
    extra_lines = [
        "  datasets:",
        f"    - folder_path: \"{data_dir}\" # Путь к картинкам mng oks bl",
        "      caption_ext: \"txt\" # Описания из текстовых файлов",
        "      cache_latents_to_disk: true # Кэш латентов на диск Z",
        "  train:",
        "    batch_size: 1 # Размер батча строго единица",
        "    steps: 2000 # Длина прыжка — 2000 шагов",
        "    gradient_accumulation: 1 # Накопление градиентов отключено",
        "    gradient_checkpointing: true # Защита VRAM от переполнения",
        "    optimizer: \"adamw8bit\" # 8-битный оптимизатор Adam",
        "    dtype: bf16 # Вычисления строго в bfloat16",
    ]
    return current_yaml + "\n".join(extra_lines) + "\n"

def assemble_and_save_monolith(config_dir: Path, data_dir: Path):
    """Сборка параметров плавки с изоляцией T5XXL и запись в бункер"""

    # Ремонт цепи: экранируем слэши пути датасета ЗА ПРЕДЕЛАМИ f-строки
    safe_data_path = str(data_dir).replace('\\', '\\\\')

    # Реальные пути к нашей SVD-красотке и VAE из трюма сканирования
    model_path = "Z:\\\\flowch\\\\models_core\\\\transformer\\\\chroma-unlocked-v50-annealed float8 e4m3fn learned svd."
```

```

vae_path = "Z:\\\\flowch\\\\models_core\\\\vae\\\\flux-vae-bf16.safetensors"

yaml_content = f"""# =====
# БОЕВОЙ ПОЛЕТНЫЙ ЛИСТ: КУЗНЯ CHROMA (CORE SPEC V03)
# БУНТ ПРИНЯТ. ТЕКСТОВЫЙ ЭНКОДЕР АМПУТИРОВАН ИЗ VRAM ДЛЯ ЗАЩИТЫ ОТ ОВЕРСВАПА
# =====

job: extension
config:
  name: "chroma_mng oks_v1"
  process:
    - type: 'sd_trainer'
      training_folder: "Z:\\\\flowch\\\\run"
      device: cuda:0

    # Сектор 1: Настройки LoRA-матрицы по канону Kohya
    network:
      type: "lora"
      linear: 16
      linear_alpha: 16

    # Сектор 2: Датасет и тотальный пре-кэш текстового пространства на диск Z
    datasets:
      - folder_path: "{safe_data_path}"
        caption_ext: "txt"
        cache_latents_to_disk: true # Кэш геометрии
        cache_text_encoder_to_disk: true # ЗАКОН ВЫЖЕННОЙ ЗЕМЛИ ДЛЯ T5XXL

    # Сектор 3: Контур стабилизации и оптимизаторы
    train:
      batch_size: 1
      steps: 2000
      gradient_checkpointing: true # Autograd защита WDDM
      optimizer: "adamw8bit"
      dtype: bf16 # Вычисления строго в bfloat16 (Защита AdaLN)

    # Сектор 4: Изолированное FP8-ядро Метрополии
    model:
      name_or_path: "{model_path}"
      vae_path: "{vae_path}"
      quantize: true # Включение TorchAO Hooks фильтрации
      train_text_encoder: false # Полное вымывание T5 из VRAM

    # Сектор 5: Контур безопасного тест-сэмплинга (без шага 0)
    sample:
      sample_every: 250
      sample_start_step: 250 # Защита от краша на старте
      width: 1024
      height: 1024

"""

target_file = config_dir / "train_chroma_mng oks.yaml"
target_file.write_text(yaml_content.strip(), encoding='utf-8')
log(f"Реактор перенастроен! Чистый YAML запечатан в бункере: {target_file}")

if __name__ == "__main__":
    log("Запуск генератора кузнечного конфига...")
    # 1. Запуск новой структуры окружения из Блока 6
    root, data, config_dir = init_core_structure()
    # 2. Прямой вызов сборщика монолита без промежуточного мусора
    assemble_and_save_monolith(config_dir, data)

```